



GeoSoft: A Language for Soft Querying Features Within GeoJSON Information Layers

Paolo Fosci¹ , Stefania Marrara² , and Giuseppe Psaila¹

¹ University of Bergamo, DIGIP, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci, giuseppe.psaila}@unibg.it

² Cefriel, Viale Sarca 226, 20126 Milan, Italy
Stefania.Marrara@cefriel.com

Abstract. *GeoJSON* has become one of the most popular format for representing spatial information. Its popularity is due to the fact that it relies on *JSON* as hosting syntactic structure. Currently, querying in an effective way a *GeoJSON* document, to extract features of interests, can be hard, for various reasons.

In this paper, we propose a domain-specific language named *GeoSoft*: it is a high-level tool that hides details of the *GeoJSON* format, which enables soft querying of features, to express imprecise queries. The paper shows that a *GeoSoft* query can be effectively and automatically translated into a *J-CO-QL* script, which is executed by the *J-CO* Framework, i.e., the execution engine we chose for *GeoSoft*.

Keywords: GeoJSON documents · Soft querying of GeoJSON documents · Fuzzy *J-CO-QL* queries

1 Introduction

In the era of Big Data and Open Data, the *JSON* format has become the *de-facto* standard for representing and sharing data and documents over the Internet. Indeed, it is possible to obtain *JSON* documents from Open Data portals, as well as from APIs (Application Programming Interfaces) of social media and, more in general, Web Services.

The GIS (Geographical Information Systems) community has defined *GeoJSON*¹: it is an international standard whose goal is to describe “spatial information layers”. In Geography, an information layer is a set of geographical (or spatial) entities, which are located on the Earth globe according to a determined set of spatial coordinates. GIS tools are able to import and export *GeoJSON* documents; furthermore, public administrations publish authoritative geographical data in form of *GeoJSON* documents on their Open Data portals.

The relevance of *GeoJSON* in the current panorama is a matter of fact: *GeoJSON* layers are used by analysts to get information about a given territory. Conceptually, a *GeoJSON* layer is a set of “features”, where each feature has its set of “properties” and its “geometry”. Consequently, the idea of “querying” features in a *GeoJSON*

¹ <https://geojson.org/>, accessed on 30/10/2021.

layer is straightforward. However, it is not easy to perform: GIS tools does not provide query languages as people used to work on relational databases could think; relational databases extended with support for geographical data (such as *PostgreSQL* extended with *PostGIS*) usually do not natively ingest *GeoJSON* documents (consequently, long and complex preprocessing activities are needed); finally, *NoSQL* document stores (such as *MongoDB*) deal with *GeoJSON* documents as pure *JSON* documents, forcing users to write preprocessing queries to extract features from *GeoJSON* information layers. Consequently, we guessed that if users wishing to query features in a *GeoJSON* document were provided with a DSL (Domain-Specific Language) for querying *GeoJSON* features, they would greatly increase their efficiency and effectiveness.

What characteristics should be provided by such a query language for *GeoJSON* layers? Moving from our previous research work on soft querying in relational databases [5, 6], we argued that a soft approach is more favorable, because very often it is not possible to know the data set to query in a precise way, in particular as far as values of numerical properties are concerned. Indeed, the soft approach allows analysts to perform imprecise matching. A second important decision we had to make was the level of the language: usually, query languages provided by *NoSQL JSON* document stores are too low-level, because they must be generic. In contrast, we decided for a high-level query language, specifically designed for *GeoJSON* documents, which provides a simplified view of the problem of querying features within *GeoJSON* documents. The first result was the language for soft querying *GeoJSON* documents that we presented in [16].

In this paper, we present *GeoSoft*: it is the first assessment of the language for soft querying *GeoJSON* layers proposed in [16]. Specifically, we have consolidated the syntax, in order to make it practically feasible to be applied in a real context. Second, we implemented a prototype translator on top of the *J-CO* Framework [19, 28] and its query language named *J-CO-QL*; the *J-CO* Framework is a research framework under development at University of Bergamo, aimed to gather, transform, integrate and query possibly-large collections of *JSON* documents, which can possibly describe spatial entities (through a geo-tagging); currently, the *J-CO-QL* language has been extended with partial support for evaluating fuzzy sets on *JSON* documents, so as to enable soft querying on them. Third, this paper presents a novel case study, based on EU NUTS (acronym from the french *Nomenclature des Unités Territoriales Statistiques* that is a hierarchical system for dividing up the economic territory of the European Union for statistical purposes)² to illustrate the potential application of *GeoSoft*.

The paper is organized as follows. Section 2 discusses the relevant related works. Section 3 presents the *J-CO* Framework, its main characteristics and how *J-CO-QL* (its query language) actually supports soft querying on *JSON* documents. Section 4 introduces the *GeoJSON* format; furthermore, a running example is introduced. Section 5 actually presents the *GeoSoft* language, by means of the running example introduced in Sect. 4. Section 6 presents the translation technique we implemented in the prototype translator, which converts a *GeoSoft* query into a *J-CO-QL* query. Finally, Sect. 7 draws the conclusions and sketches future work.

² <https://ec.europa.eu/eurostat/web/nuts/background>, accessed on 30/10/2021.

2 Related Work

Talking about a flexible language for geographical *JSON* documents, we need to evaluate three different research topics: languages to express *JSON* data, language to express flexible queries and, finally, languages specifically designed for geographical data. The first two topics have been covered in our previous work [19], here we just summarize the most important contributions.

With the rise of *JSON* as new *de-facto* standard for data exchange on the Web, many systems specifically designed to store *JSON* documents have been designed as subclass of *NoSQL* databases; the most popular is *MongoDB*³, which is used for storing and querying large collections of small *JSON* documents. Another important example is *CouchDB*⁴ [21], which has been chosen as the underlying database engine for the *HyperLedger Fabric* Blockchain platform [8]. As a consequence, many query languages for *JSON* data sets have been proposed by the database community. The most popular ones are: *Jaql* [25], *SQL++* [26], *JSONiq* [14] and the query language provided by *MongoDB* [11].

In the domain of soft query languages, there are two main approaches: the first one extends the relational data model towards a fuzzy database model (as in *FSQL* [20]). The second approach does not modify the relational data model but extends the *SQL* query language with the capability to fuzzy query a conventional relational database (such as *SoftSQL* [6] and *SQLf* [7]).

The topic of this paper is closely related to the area of Geographical Information Systems; in this domain, several proposals have been made, especially for data storage, indexing and query optimization [1]. GIS are supposed to store huge amounts of complex data, so as to graphically represent them. Many extensions of the *SQL* language have been proposed [12, 13]. These extensions are necessary in order to allow Data Base Management Systems (DBMS) to store and retrieve spatial information. However, these languages are not designed for end-users, due to the complexity of the *SQL* language. To overcome this issue, other proposals include tabular approaches [30] defined as extensions of QBE (Query By Example) [22]), graphical languages [23], visual languages [24], and *hypermaps* [10] (integrated into hypermedia techniques [2]).

If the geographical information is stored as *GeoJSON* documents, most systems provide traditional *JSON* query languages to access them. Talking about query languages that are specifically targeted to *GeoJSON*, the number of available proposals is very low. We can cite *GeoPQLJ* [15]: it is a pictorial query language that allows users to formulate their queries by using drawing facilities, so as to help users correctly interpret the query syntax and semantics on the basis of its underlying algebra. *GeoPQL* has been conceived as a stand-alone GIS client which uses the *ESRI* Library⁵.

³ *MongoDB*. 2021. Available online: <https://www.mongodb.com/> accessed on 30/10/2021.

⁴ *CouchDb*. 2021. Available online: <https://couchdb.apache.org/> accessed on 30/10/2021.

⁵ <https://www.esri.com/arcgis-blog/products/arcgis-hub/announcements/welcome-to-the-content-library/> accessed on 30/10/2021.

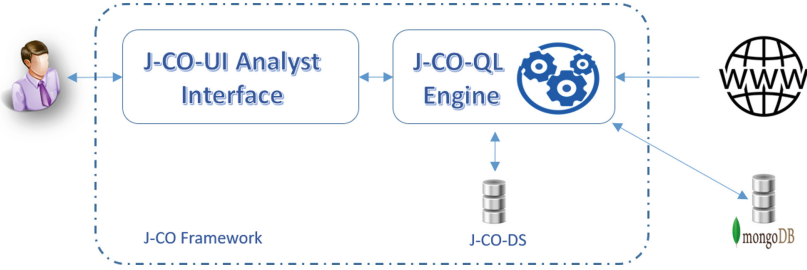


Fig. 1. Components of the *J-CO* framework.

3 Soft Querying *JSON* Documents: The *J-CO* Framework

In this section, we introduce the background on which our work relies, i.e., the *J-CO* Framework and its main features, in particular as far as soft querying of *JSON* documents is concerned.

The *J-CO* Framework is the result of a research work [3, 4, 27, 28] conducted at University of Bergamo, towards the definition of a tool able to retrieve, integrate, transform, manage and query possibly-large collections of *JSON* documents either stored in *JSON* document store or directly provided by Web sources.

The core of the framework is *J-CO-QL*, a novel query language specifically designed to query heterogeneous *JSON* data sets, by natively supporting geo-spatial aggregations of geo-tagged documents. In [17–19, 29], we proposed to extend the language in order to support fuzzy sets and soft querying.

The *J-CO* Framework is composed by several software tools, as depicted in Fig. 1. We report a brief summary of them (the interested reader can refer to [28]).

The *J-CO-QL Engine* actually processes *J-CO-QL* scripts. It is able to access Web sources and can connect to *JSON* stores like *MongoDB*.

J-CO-DS is a *JSON* document store developed within the *J-CO* Framework, which can store very-large single *JSON* documents.

Finally, *J-CO-UI* is the user interface of the framework: users can use it to interact with the *J-CO-QL Engine*.

Fuzzy Sets. Fuzzy-Set Theory was introduced by Zadeh [31]. Let us consider a non-empty universe X , either finite or infinite. A fuzzy set $A \in X$ is a mapping $A : X \rightarrow [0, 1]$. The value $A(x)$ is referred to as the *membership degree* of the element x to the fuzzy set A . In other words, given a fuzzy set $A \in X$, a membership function $A(x)$ associates each element $x \in X$ with a real number in the interval $[0, 1]$; given x , the value $A(x)$ is the degree of membership of x to A .

If $A(x) = 1$, x fully belongs to A . If $A(x) = 0$, x does not belong at all to A . An intermediate value $0 < A(x) < 1$ means that x belongs to A in a partial way (i.e., $A(x) = 0.9$ means that x does not completely belong to A). For example, if A stands for “old people” and x_1 and x_2 are two persons, the fact that $A(x_1) = 0.9$ and $A(x_2) = 0.3$ denotes that neither x_1 nor x_2 are completely old, but certainly x_1 is older than x_2 .

Consequently, fuzzy sets express vague or imprecise concepts on real-world entities, as well as they express vague or imprecise relationships among real-world entities.

J-CO-QL works on collections of *JSON* documents; consequently, given a *JSON* document d , it provides constructs to evaluate its membership degrees to several fuzzy sets $A_1, \dots, A_n$ at the same time, i.e., $A_1(d), \dots, A_n(d)$.

Fuzzy Operators. In order to evaluate if a *JSON* document belongs to a given fuzzy set, *J-CO-QL* allows for defining *Fuzzy Operators*.

Listing 1 reports the *J-CO-QL* script that defines two fuzzy operators, named `Has DECREASING Population` and `Has INCREASING GDP`. Hereafter, we briefly describe their definitions.

Listing 1. Defining Fuzzy Operators.

```

1: CREATE FUZZY OPERATOR Has DECREASING Population
  PARAMETERS
    Pop1 TYPE Float,
    Pop2 TYPE Float
  PRECONDITION
    Pop1 > 0
    AND Pop2 > 0
  EVALUATE 100*(Pop2-Pop1) / Pop1
  POLYLINE
    [ (-2.0, 1.0),
      (-1.0, 0.7),
      (-0.5, 0.1),
      ( 0.0, 0.0) ];

2: CREATE FUZZY OPERATOR Has INCREASING GDP
  PARAMETERS
    GDP1 TYPE Float,
    GDP2 TYPE Float
  PRECONDITION
    GDP1 > 0
    AND GDP2 > 0
  EVALUATE 100*(GDP2-GDP1) / GDP1
  POLYLINE
    [ ( 0, 0.0),
      ( 2, 0.1),
      ( 4, 0.5),
      ( 7, 0.8),
      (10, 0.9),
      (15, 1.0) ];

```

Consider the first fuzzy operator, named `Has DECREASING Population`. Given the population in two consecutive years (referring to the same area), it expresses the fact that the population decreases.

The `PARAMETERS` clause specifies the two formal parameters, named `Pop1` and `Pop2`, whose actual values are the amounts of population to compare.

The `PRECONDITION` clause specifies a condition on actual parameters that must be met before proceedings with the evaluation of the operator. If it is not met, an error signal is raised and the evaluation is stopped.

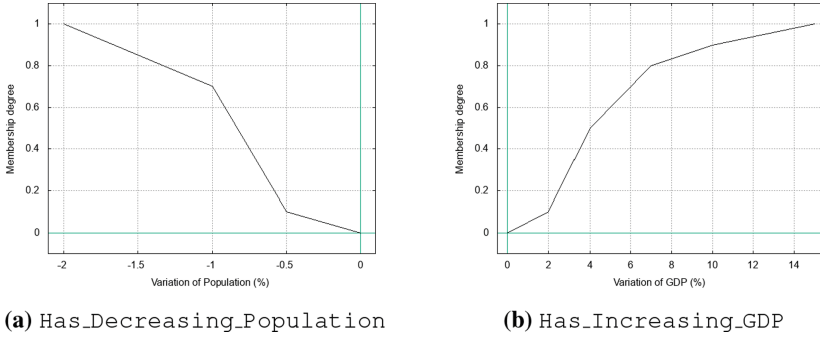


Fig. 2. Membership functions of fuzzy operators in Listing 1.

The `EVALUATE` clause is used to specify an expression to evaluate on the basis of actual parameters; in this case, the expression computes the percentage of variation of population, which can be either positive or negative.

The `POLYLINE` clause provides the coordinates of vertices of a polyline function actually used to evaluate the membership degree of the operator. The polyline defined in the `Has_Decreasing_Population` operator is depicted in Fig. 2a: the value obtained by evaluating the expression in the `EVALUATE` clause is used as x coordinate; the corresponding y value on the polyline is the output membership degree. Notice that the percentage of population variation could be less than -2% , as well as it could be greater than 0% : in these cases, the left extreme value (i.e., 1) and, respectively, the right extreme value (i.e., 0) are returned as membership values.

The second fuzzy operator is named `Has_Increasing_GDP`. Its goal is to evaluate positive variations of GDP (Gross Domestic Product) of a region.

Similarly to the previous operator, two formal parameters are defined, named `GDP1` and `GDP2`; the condition in the `PRECONDITION` clause ensures that the operator is evaluated only for positive values of the parameters.

The `EVALUATE` clause defines the expression that computes the percentage of GDP variation. Then, the `POLYLINE` clause defines the membership function, which is depicted in Fig. 2b. The reader can see that the two polylines are mirrored: the left one must be 1 for negative variations of population, while the right one must be 1 for positive variations of GDP.

Soft Querying a Collection of Documents. Let us suppose that we have a collection of *JSON* documents, named `NUTSCollection`, stored within the `NUTS_Info` database (managed by a *MongoDB* server), where each document describes a level-3 European NUTS (corresponding to a province or a county, depending on the country). An example of document (describing a NUTS) is shown in Fig. 3a. It reports the code (`NUTSCode`), the name (`NUTSName`), the level (`NUTSLevel`), the country code (`CountryCode`), the population in 2017 (`population2018`) and in 2018 (`population018`) and the GDP in 2017 (`GDP2017`) and in 2018 (`GDP2018`).

Listing 2 reports the second part of the *J-CO-QL* script, whose goal is to select those documents that describe a NUTS for which a significant reduction of population was

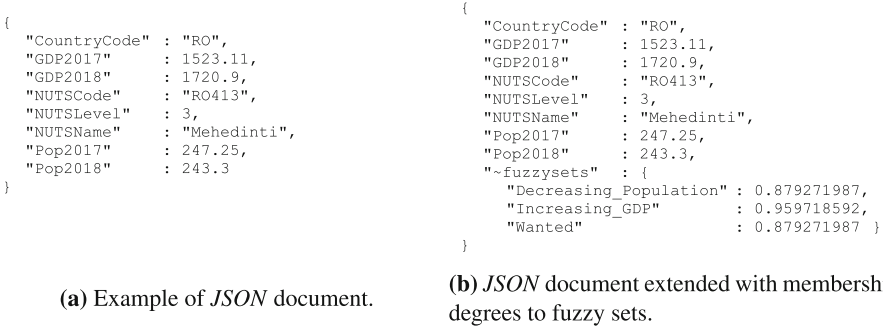


Fig. 3. Example of *JSON* document describing a NUTS (a), later extended (b) with membership degrees to fuzzy sets.

```

{
  "CountryCode" : "RO",
  "NUTSCode" : "RO413",
  "NUTSLevel" : 3,
  "NUTSName" : "Mehedinti",
  "rank" : 0.879271987
}

```

Fig. 4. Example of document obtained by the soft query.

registered, jointly with a significant increase of GDP, from 2017 to 2018. To illustrate the script, we have to introduce both the data model and the execution model of *J-CO-QL* scripts.

- *Data Model.* *J-CO-QL* works on standard *JSON* documents; however, root level fields whose name begins with the “~” character play a special role. For example, the `~fuzzysets` field works as a map $fsn \rightarrow md$, where fsn is a fuzzy-set name and md is the corresponding membership degree; this way, the degrees of membership of a document to multiple fuzzy sets can be simultaneously represented. The model encompasses the `~geometry` field too, for representing spatial geometries [3, 28].
- *Execution Model.* A clear and linear execution model has been defined. The query (or script) is a sequence of instructions. The *query-process state* s contains the so-called *temporary collection* $s.tc$: an instruction i_j works on the $s_{(j-1)}$ query-process state and generates a new query-process state s_j , meaning that i_j receives the $s_{(j-1)}.tc$ temporary collection as input and generates the new $s_j.tc$ temporary collection as output; in the initial state s_0 , it is $s_0.tc = \emptyset$. Consequently, instructions are *pipelined*, so as the user can write them following the natural flow with which they are thought.

We can now present the instructions of the script in Listing 2.

Listing 2. Soft querying with *J-CO-QL*.

```

3: USE DB NUTS_Info
   ON SERVER MongoDB 'http://127.0.0.1:27017';

4: GET COLLECTION NUTSCollection@NUTS_Info;

5: FILTER
   CASE
     WHERE WITH .Pop2017, .Pop2018, .GDP2017, .GDP2018
       CHECK FOR FUZZY SET Decreasing_Population
         USING Has_Decreasing_Population (.Pop2017, .Pop2018)
       CHECK FOR FUZZY SET Increasing_GDP
         USING Has_Increasing_GDP (.GDP2017, .GDP2018)
       CHECK FOR FUZZY SET Wanted
         USING ( Decreasing_Population AND Increasing_GDP )
       ALPHA-CUT 0.8 ON Wanted
     DROP OTHERS;

6: FILTER
   CASE
     WHERE KNOWN FUZZY SETS Wanted
     GENERATE
       { .NUTSName      : .NUTSName,
         .NUTSCode      : .NUTSCode,
         .NUTSLevel     : .NUTSLevel,
         .CountryCode   : .CountryCode,
         .rank           : MEMBERSHIP_OF (Wanted) }
     DROPPING ALL FUZZY SETS
     DROP OTHERS;

7: SAVE AS DetectedNUTS@NUTS_Info;

```

- On line 3, the `USE DB` instruction connects the *J-CO-QL Engine* to a *MongoDB* database named `NUTS_Info`, managed by a *MongoDB* server running on the same server where the *J-CO-QL Engine* is running.
- The `GET COLLECTION` instruction on line 4 retrieves the collection named `NUTSCollection` from the `NUTS_Info` database, so as to make it the new temporary collection. Figure 3a reports an example document in the collection (remember that each document describes a level-3 NUTS).
- The `FILTER` instruction on line 5 actually performs the soft querying of *JSON* documents in the current temporary collection.

Given a document d in the temporary collection, the `CASE WHERE` selection clause evaluates if it has the fields specified in the `WITH` predicate; if d satisfies the condition, it is considered for further processing. Notice the dot notation used to refer to fields: the initial dot (e.g. `.Pop2017`) means that the field is at the root level of d . The three `CHECK FOR FUZZY SET` clauses evaluate the membership degrees of d to three fuzzy sets, named `Decreasing_Population`, `Increasing_GDP` and `Wanted`.

The first `CHECK FOR FUZZY SET` clause applies the fuzzy operator named `Has_Decreasing_Population` on d 's fields in the `USING` sub-clause, which is a fuzzy condition. The resulting membership degree is associated to d by adding a new field named `~fuzzysets`, as depicted in Fig. 3b: within it, the nested field named `Decreasing_Population` denotes the membership degree of d to the

Decreasing_Population fuzzy set.

The second CHECK FOR FUZZY SET clause behaves similarly: it evaluates the membership degree of d to the fuzzy set named Increasing_GDP, through the Has_Increasing_GDP fuzzy operator.

The last CHECK FOR FUZZY SET clause evaluates the membership of d to the Wanted fuzzy set, by means of a soft condition based on the AND operator, i.e., the minimum membership degree to the Decreasing_Population and Increasing_GDP fuzzy sets is considered as membership degree to the Wanted fuzzy set. The final structure of d is reported in Fig. 3b.

At this point, the ALPHA-CUT clause specifies a minimum threshold on the Wanted fuzzy set: if d has a membership degree that is less than the threshold of 0.8, it is not inserted into the output temporary collection.

Notice the final DROP OTHERS option, which discards all documents that do not meet the CASE WHERE condition from the output temporary collection.

- The FILTER instruction on line 6 has to *de-fuzzify* the selected documents and add a new field to the original structure of documents. This is done by the GENERATE action, which specifies the novel structure of the documents: notice the new rank field, whose value is the membership degree to the Wanted fuzzy set, obtained by means of the MEMBERSHIP_OF function. Finally, all membership degrees to fuzzy sets are discarded from documents by the DROPPING ALL FUZZY SETS option. An example of document in the output collection is depicted in Fig. 4.
- The SAVE AS instruction on line 7 saves the final temporary collection into the DetectedNUTS collection in the NUTS_Info database.

J-CO-QL provides many other statements, in particular in the rest of the paper we will make use of the EXPAND and GROUP statements (they will be described when they will be used). Nevertheless, the script presented in this section should have clarified how *J-CO-QL* supports fuzzy querying on JSON documents.

4 Introducing GeoJSON and the Running Example

This section briefly introduces the *GeoJSON* format; then, it presents the data set adopted as running example to show how the *GeoSoft* language works.

4.1 GeoJSON

GeoJSON is an interchange format for spatial data. It adopts open standards and it is used to represent geographic features and their non-spatial properties. *GeoJSON* is based on the JavaScript Object Notation (*JSON*) as host syntactic format, due to its simplicity and wide diffusion: this characteristic allows for processing *GeoJSON* documents as any other *JSON* document. Typically, it is used for encoding a variety of geographical information layers, i.e., aggregations of spatial features that provide a specific type of spatial information (e.g., roads, buildings, rivers, and so on). It is independent of any geographic coordinate reference system; however, most of *GeoJSON* documents implicitly adopt the World Geodesic System 1984 (WGS-84) and decimal degree units.

Currently, *GeoJSON* has become very popular in many GIS tools and services related to web mapping. It is actually used by Web applications as a standard format for exchanging geographical information layers in client-server communication; this is due to the fact that a *GeoJSON* document is a *JSON* document, so client-side libraries and platforms for web mapping written by means of the JavaScript programming language (such as *Leaflet*⁶, *CARTO*⁷, and *Turf.js*⁸) easily process it.

Another advantage of *GeoJSON* is that it is a human-readable format, since it is a plain-text format; however, this characteristic makes it verbose, because property names are repeated for each single feature, even though they all have the same properties. As an effect, *GeoJSON* documents can become much larger if compared to other formats for representing spatial data, such as *Shapefile*⁹ and *GeoPackage*¹⁰.

To illustrate the structure of a *GeoJSON* document, consider the document reported in Fig. 5a. Hereafter, a brief introduction to its structure is reported (more details can be found in [9]).

- A *GeoJSON* document is a unique, single and possibly-giant *JSON* document. At the root level, two mandatory fields are present: the *type* field has the "FeatureCollection" value, in order to denote that this is the root level (since an information layer is a collection of spatial features); the *features* field is an array of documents, each one describing a spatial feature.
- A document describing a single feature has three mandatory fields: the *type* field has the "Feature" value, in order to denote that the document actually describes a spatial feature; the *properties* field is a nested *JSON* document that reports non-spatial properties of the feature; the *geometry* field actually denotes the spatial property (also named "geometry") of the spatial feature.
- The *properties* field has not a specific structure; it is a list of possibly-complex properties, based on the *JSON* format.
- The *geometry* field describes the geometry of the spatial feature. The format encompasses points, line-strings, polygons and mixed geometries.

4.2 Running Example

In order to illustrate the goal of the *GeoSoft* language, and to let the reader understand it, a running example is adopted. The example originates from the *Eurostat* (the European Institute for Statistics) portal¹¹, which publishes many interesting data sets concerning the European Union. Among them, *GeoJSON* documents, describing European countries, are available. Specifically, we downloaded the *GeoJSON* layer¹² that describes the

⁶ <https://leafletjs.com/examples/geojson/>, accessed on 30/10/2021.

⁷ <https://carto.com/developers/carto-vl/guides/add-data-sources/>, accessed on 30/10/2021.

⁸ <https://turfjs.org/>, accessed on 30/10/2021.

⁹ <https://www.statsilk.com/maps/convert-esri-shapefile-map-geojson-format>, accessed on 30/10/2021.

¹⁰ <https://www.geopackage.org/guidance/modeling.html>, accessed on 30/10/2021.

¹¹ <https://ec.europa.eu/eurostat>, accessed on 30/10/2021.

¹² <https://ec.europa.eu/eurostat/web/gisco/geodata/reference-data/administrative-units-statistical-units/nuts>, accessed on 30/10/2021. We opted for the 1:3Million scale since the *GeoJSON* layer at 1:1Million scale was too big to be stored into a *MongoDB* database.

```

{
  "type" : "FeatureCollection",
  "features" : [
    {
      "type" : "Feature"
      "properties" : {
        "CountryCode" : "BG",
        "GDP2017" : 1011.36,
        "GDP2018" : 1260.6,
        "NUTSCode" : "BG313",
        "NUTSLevel" : 3,
        "NUTSName" : "Vratsa",
        "Pop2017" : 167.19,
        "Pop2018" : 164.1
      },
      "geometry" : {
        "type" : "Polygon",
        "coordinates" : [
          [
            [ 24.323523, 43.697686],
            [ 24.30067, 43.647582],
            ... ] ]
        }
      }
    },
    { ... }, ..., { ... }
  ]
}

```

(a) Excerpt of the reference
GeoJSON document.

```

{
  "type" : "FeatureCollection",
  "features" : [
    {
      "type" : "Feature"
      "properties" : {
        "CountryCode" : "BG",
        "NUTSCode" : "BG313",
        "NUTSLevel" : 3,
        "NUTSName" : "Vratsa",
        "rank" : 0.954458997
      },
      "geometry" : {
        "type" : "Polygon",
        "coordinates" : [
          [
            [ 24.323523, 43.697686],
            [ 24.30067, 43.647582],
            ... ] ]
        }
      }
    },
    { ... }, ..., { ... }
  ]
}

```

(b) Excerpt of the output
GeoJSON document.

Fig. 5. *GeoJSON* documents for the running example, which describe (a) all level-3 NUTS in Europe and (b) those selected by the sample *GeoSoft* query.

geometries of the level-3 NUTS; they are depicted on the left side of Fig. 6. For each NUTS, the downloaded *GeoJSON* layer reports only registry properties (such as code, name, level and country code), while data such as population¹³ and GDP¹⁴ were provided in different pages as CSV (Comma-Separated Values) files, that were converted into *JSON* documents¹⁵. By means of a *J-CO-QL* script, which we do not report here since it is out of the scope of this paper, we were able to integrate the basic *GeoJSON* layer with the additional sources, to obtain a new *GeoJSON* layer that reports registry properties, population and GDP in 2017 and 2018 of level-3 NUTS. An excerpt of the reference *GeoJSON* document is reported in Fig. 5a.

5 *GeoSoft*: An SQL-like Language

Consider the *GeoJSON* document reported in Fig. 5a. A *GeoJSON* document can be viewed as a container of features. So, it is straightforward to guess that these features could be queried, so as to obtain a novel *GeoJSON* document (i.e., a novel container of features). The availability of a query language to perform such a query without the need to deal with technicalities concerned with processing of *GeoJSON* documents would significantly help analysts. However, we make a step forward: we consider not only querying but *soft querying a GeoJSON document*; we mean searching for features that meet a qualitative condition, in order to get a new *GeoJSON* document, in which

¹³ https://ec.europa.eu/eurostat/databrowser/view/nama_10r_3popgdp/default/table, accessed on 30/10/2021.

¹⁴ [https://ec.europa.eu/eurostat/databrowser/view/NAMA_10_GDP\\$DEFAULTVIEW/default/table](https://ec.europa.eu/eurostat/databrowser/view/NAMA_10_GDP$DEFAULTVIEW/default/table), accessed on 30/10/2021.

¹⁵ for this purpose we used: <https://www.convertcsv.com/csv-to-json.htm>, accessed on 30/10/2021.

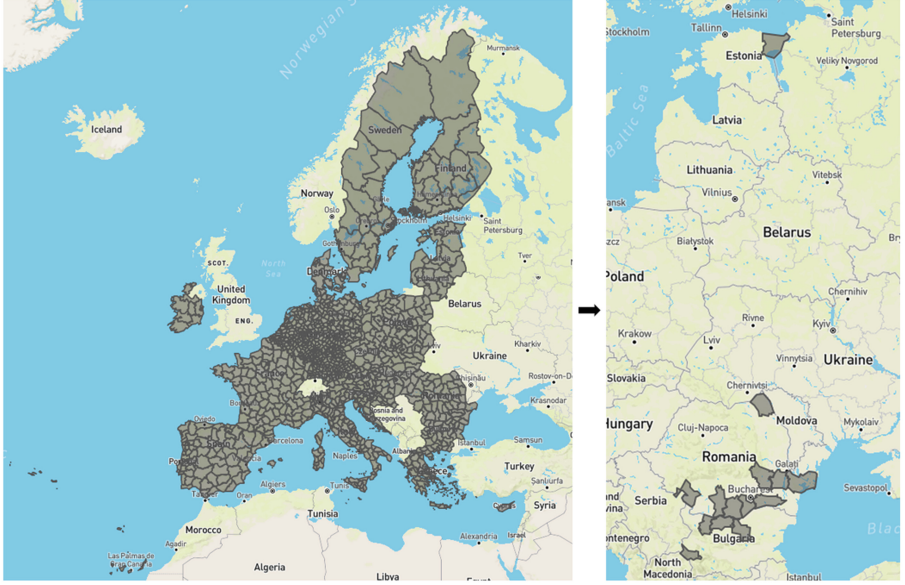


Fig. 6. On the left, the map depicting all EU Level-3 NUTS; On the right, the map depicting only the selected Level-3 NUTS.

features might be sorted in reverse order of importance (as far as the qualitative selection condition is concerned).

5.1 Semantic Model

The language we propose is named *GeoSoft*. It is strongly inspired by the well known syntax of the SQL *SELECT* statement, because it is quite familiar to analysts. However, we are introducing soft concepts to work on a crisp context. For this reason, we have to clearly distinguish the *external* and the *internal* semantic models.

External Semantic Model. *GeoSoft* is designed to work on *GeoJSON* documents, so as to generate new *GeoJSON* documents. Thus, the external semantic model (i.e., independent of what happens inside the query) is the following one.

- A *GeoSoft* query gsq can be seen as a function

$$gsq : GJ \rightarrow GJ$$

where GJ is the domain of (crisp) *GeoJSON* documents.

- A (crisp) *GeoJSON* document can be seen as a “set of features” (if we ignore syntactic aspects), where a feature can be denoted as a tuple $f = \langle \text{properties}, \text{geometry} \rangle$ where *properties* and *geometry* are defined as in the *GeoJSON* standard (see Sect. 4.1).

Listing 3. Simple soft query in *GeoSoft*.

```

SELECT .CountryCode, .NUTSCODE, .NUTSLevel, .NUTSName
FROM NUTSCollection_GeoJSON@NUTS_Info
WHERE Decreasing_Population AND Increasing_GDP
FOR FUZZY SET Wanted
ALPHA-CUT Wanted: 0.8
ADD MEMBERSHIP_OF Wanted AS .rank
ORDER BY .rank DESC
SAVE AS DetectedNUTS_Geojson@NUTS_Info;

```

Internal Semantic Model. In order to meet the external semantic model, soft concepts must be confined within the *gsq* *GeoSoft* query. Thus, it is necessary to introduce the concept of *internal query*, denoted as *iq*, which works on a different domain than *GJ* (the domain of crisp *GeoJSON* documents).

- A *soft feature* $\bar{f}$ is a feature for which its membership degrees to some fuzzy sets are known. A soft feature is defined as a tuple

$$\bar{f} = \langle \text{properties}, \text{geometry}, \text{fuzzysets} \rangle$$

where *properties* and *geometry* are defined as in the *GeoJSON* standard. Each feature can belong to several fuzzy sets with a specific membership degree (the membership degree to each fuzzy set denotes the degree with which the feature belongs to the fuzzy set). In the $\bar{f}$ tuple, the *fuzzysets* field is a key/value map, that associates a fuzzy set name *fsn* to the membership degree of the $\bar{f}$ feature to the fuzzy set *fsn*.

- A *Soft GeoJSON* document $\bar{d}$ is a “set of soft features”. The domain of *Soft GeoJSON* documents is denoted as *SGJ*.
- An *Internal GeoSoft* query *iq* can be seen as a function

$$iq : SGJ \rightarrow SGJ$$

but *SGJ* exists only within a *GeoSoft* query *gsq*. Consequently, by introducing two functions named *get* and *set*, it is possible to join the two semantic models.

It is:

$$gsq = set(iq(get(d)))$$

where *get* : *GJ* → *SGJ* translates a *GeoJSON* document $d \in GJ$ into a *Soft GeoJSON* document $\bar{d} \in SGJ$ (*fuzzification*), while *set* : *SGJ* → *GJ* translates a *soft GeoJSON* document $\bar{d} \in SGJ$ into a (crisp) *GeoJSON* document $d \in GJ$ (*de-fuzzification*).

5.2 Queries

Based on the running example (Sect. 4.2), a sample qualitative query may be: *extract those NUTS (features) that, between 2017 and 2018, registered a significant reduction of population and a significant increment of GDP.*

Listing 3 shows the query as it would be if two linguistic predicates named *Decreasing_Population* and *Increased_GDP* were available.

The idea is the following: if we are able to evaluate the two linguistic predicates on each feature, we can evaluate a compound `Wanted` linguistic predicate; the membership value associated to this linguistic predicate denotes the degree with which the single feature matches the compound linguistic condition. Let us describe our idea in details.

- The `FROM` clause specifies the collection in the *JSON* document database containing the source *GeoJSON* document.
- The `WHERE` clause specifies the compound linguistic condition, by evaluating the membership degree for each single feature.

The last part of the clause, i.e., `FOR FUZZY SET Wanted`, gives a name to the fuzzy set to which the membership degree is evaluated by means of the compound linguistic condition.

Thus, the `WHERE` clause is now a “soft condition”: it computes the membership degree of each $\bar{f}$ feature to a given fuzzy set. A single term t that appears without dot notation is assumed to be a fuzzy set name and its membership degree is retrieved from the $\bar{f}.\text{fuzzysets}$ map (if it is not in the map, the 0 value is assumed as its membership degree).

- The `SELECT` clause specifies the properties of the feature to project on. Notice that property names are denoted by means of the dot notation, which refers to the $\bar{f}.\text{properties}$ field as the root.
- The `ALPHA-CUT` clause selects only those features that belong to the `Wanted` fuzzy set with membership degree no less than 0.8.
- `ADD MEMBERSHIP OF Wanted AS .rank` specifies that the membership value of the `Wanted` fuzzy set becomes a new property in features, whose name is `rank`.
- The optional `ORDER BY` clause sorts the selected features by the novel `.rank` property, in descending order. This way, the output *GeoJSON* document will show in the first positions of the `features` array those features that are most relevant w.r.t. the query.
- Finally, the `SAVE AS` clause specifies that the resulting *GeoJSON* document must be saved into the `DetectedNUTS_Geojson` collection into the *JSON* document database.

Referring to the semantic model introduced in Sect. 5.1, the *internal query iq* is composed by the `SELECT`, `FROM`, `WHERE`, `ALPHA-CUT` and `ADD MEMBERSHIP` clauses; the `ORDER BY` and `SAVE AS` clauses falls into the *set* function, that *defuzzifies* the output document.

Linguistic Predicates and Fuzzy Sets. Linguistic predicates can be interpreted in terms of fuzzy sets, i.e., the satisfaction degree of a linguistic predicate corresponds to the membership degree to a fuzzy set. So, the previous soft query is correct only if features in the source *GeoJSON* document are provided with such membership degrees. However, features in *GeoJSON* documents are not provided with membership degree to any fuzzy set (i.e., $\bar{f}.\text{fuzzysets} = \emptyset$). So, apart from assuming that the membership degree for any fuzzy-set name not appearing in $\bar{f}.\text{fuzzysets}$ is 0, it is necessary

Listing 4 Complete *GeoSoft* query.

```

SELECT .CountryCode, .NUTSCode, .NUTSLevel, .NUTSName      iq3
FROM
  (SELECT *                                                  iq2
   FROM
     (SELECT *                                              iq1
      FROM NUTSCollection_Geojson@NUTS_Info
      WHERE Has_Decreasing_Population(.Pop2017, .Pop2018)
            FOR FUZZY SET Decreasing_Population
     )
    WHERE Has_Increasing_GDP(.GDP2017, .GDP2018)
          FOR FUZZY SET Increasing_GDP
  )
WHERE Decreasing_Population AND Increasing_GDP
  FOR FUZZY SET Wanted
  ALPHA-CUT Wanted : 0.8
ADD MEMBERSHIP OF Wanted AS .rank

ORDER BY .rank DESC
SAVE AS DetectedNUTS_Geojson@NUTS_Info;

```

to understand how to provide features with membership degrees to fuzzy sets corresponding to linguistic predicates. The solution is to compute them by means of “nested *GeoSoft* queries”, allowed in the FROM clause.

Based on the above-mentioned considerations, and supposing that fuzzy operators named `Has_Decreasing_Population` and `Has_Increasing_GDP` are defined, the soft query can be written as reported in Listing 4. Hereafter, we describe it.

- The innermost internal query (that we denote as iq_1) retrieves the source *GeoJSON* document from the collection named `NUTSCollection_Geojson`. This collection is stored within the database named `NUTS_Info`.

The WHERE clause uses the `Has_Decreasing_Population` fuzzy operator to evaluate the membership degree of each single feature in the document to the `Decreasing_Population` fuzzy set.

All features $\bar{f}$ are kept, but their $\bar{f}.fuzzysets$ map now is no longer empty, since it contains the `Decreasing_Population` entry.

As specified by the `*` character in the SELECT clause, all properties in the features are kept.

- The intermediate *GeoSoft* internal query (that we denote as iq_2) in Listing 4, receives the set of features generated by the innermost query iq_1 (as specified in the FROM clause).

The WHERE clause evaluates the membership degree of a feature $\bar{f}$ to the `Increasing_GDP` fuzzy set, by means of the `Has_Increasing_GDP` fuzzy operator.

Again, the properties of the features are not changed and all features appear in the result set: simply, a new entry with key `Increasing_GDP` is now present in the $\bar{f}.fuzzysets$ map of each feature.

- The outermost *GeoSoft* internal query (that we denote as iq_3) in Listing 4 is the same as the simple internal query reported in Listing 3, apart from the fact that the `FROM` clause contains the nested query discussed above.

Linguistic predicates in Listing 3 now have become fuzzy-set names, in that they have been evaluated by the two nested queries.

Clearly, after the evaluation of the `WHERE` clause, the $\bar{f}.fuzzysets$ map of each $\bar{f}$ feature actually contains three entries; they correspond to three fuzzy-set names, which are `Decreasing_Population`, `Increasing_GDP` and `Wanted`. The last one is used by the `ALPHA-CUT` clause to select those features whose membership degree to the `Wanted` fuzzy set is no less than 0.8. Furthermore, the `ADD MEMBERSHIP OF` clause adds a new property to the ones specified in the outermost `SELECT` clause (in iq_3), by using the membership degree to the `Wanted` fuzzy set. Since this is the outermost internal query, its output must fall again into the domain of *GeoJSON* documents: as a consequence, the $\bar{f}.fuzzysets$ map disappears in the $\bar{f}$ features of the output crisp *GeoJSON* document.

The reader can notice that internal queries can be compound too. In particular, Listing 4 is based on three internal queries iq_1 (the innermost), iq_2 (the intermediate) and iq_3 (the outermost), so it is $iq(\bar{d}) = iq_3(iq_2(iq_1(\bar{d})))$.

Currently, *GeoSoft* does not provide instructions to define fuzzy operators, since we assume they are already available in the execution environment. We will address this aspect too in our future work.

The above-described *GeoSoft* query works on the input *GeoJSON* document we built for the running example, an excerpt of which is reported in Fig. 5a. It contains 1169 features describing the level-3 NUTS depicted on the left side of Fig. 6; the *GeoSoft* query generates the output *GeoJSON* document (an excerpt of which is reported in Fig. 5b) that contains 14 features, each one describing a NUTS of interest, depicted on the right side of Fig. 6. These are the provinces in which a significant reduction of population from 2017 to 2018 corresponded to a significant increment of GDP, which is an interesting phenomenon to investigate.

6 Translating *GeoSoft* into *J-CO-QL*

In this section, we show how *J-CO-QL* can be used as the execution engine for *GeoSoft* queries; we will show how a *GeoSoft* query can be translated into a *J-CO-QL* script. As an example, we still refer to the running example and to the *GeoSoft* query presented in Listing 4. We divided the *J-CO-QL* script into 3 parts. Part 1 is named *preamble*: it is reported in Listing 5 and discussed in Sect. 6.1. Part 2 is named *core*: it is reported in Listing 6 and discussed in Sect. 6.2. Part 3 is named *tail*: it is reported in Listing 7 and discussed in Sect. 6.3.

Listing 5. Translation of the *GeoSoft* query: preamble.

```

1. GET COLLECTION NUTSCollection_Geojson@NUTS_Info;

2. EXPAND
    UNPACK WITH ARRAY .features
    ARRAY .features TO .feature
    DROP OTHERS;

3. FILTER
    CASE WHERE WITH .feature.item
        GENERATE {
            .type      : .feature.item.type,
            .properties : .feature.item.properties,
            .geometry   : .feature.item.geometry }
        DROP OTHERS;

```

6.1 Preamble of the *J-CO-QL* Script

The *preamble* of the *J-CO-QL* script substantially corresponds to the *get* function introduced in Sect. 5.1 to define the semantic model of *GeoSoft*. The goal of the *preamble* is to get the input *GeoJSON* document, split each feature inside the *GeoJSON* document into a single *JSON* document, and transform each single feature document in a convenient way to be easily handled. The rationale of this sub-script is that a *GeoJSON* layer, which might include several features, is a single document, while *J-CO-QL* works on collections of documents.

Hereafter, we illustrate each single instruction in Listing 5. We refer to the *GeoSoft* query reported in Listing 4.

- The `GET COLLECTION` instruction on line 1 retrieves the content of the collection named `NUTSCollection_Geojson`, which is stored within the database named `NUTS_Info`. The collection name is specified in the `FROM` clause of the innermost internal query iq_1 presented in Listing 4. Notice that the collection contains only one document, that is a *GeoJSON* document, as the *temporary collection* generated by the instruction (in the future, we will address the situation in which several *GeoJSON* documents are stored in the same database collection).
- The `EXPAND` instruction on Line 2 unnests features from within the *GeoJSON* document in the temporary collection. Specifically, the `UNPACK` condition selects documents containing an array field named `features`. The `ARRAY` clause specifies that items in the array field named `features` must be unnested: a new document for each item is generated; the `TO` keyword specifies that the unnested item must be included inside a structured field named `feature`. In each new generated document, the `feature` field contains two sub-fields: `position` denotes the position of the unnested item in the source array; `item` actually reports the content of the unnested item. The final `DROP OTHERS` clause, which in this case is ineffective, discards all documents in the input temporary collection that do not match the selection condition.
- The `FILTER` instruction on line 3 transforms the documents in the temporary collection generated by the `EXPAND` instruction on line 2, in order to give them a simplified structure, suitable for the next part of the script.

Listing 6. Translation of the *GeoSoft* query: core.

4. FILTER	<i>iq₁</i>
CASE WHERE WITH .properties.Pop2017, .properties.Pop2018 CHECK FOR FUZZY SET Decreasing_Population USING Has_Decreasing_Population(.properties.Pop2017, .properties.Pop2018) DROP OTHERS;	
5. FILTER	<i>iq₂</i>
CASE WHERE WITH .properties.GDP2017, .properties.GDP2018 CHECK FOR FUZZY SET Increasing_GDP USING Has_Increasing_GDP(.properties.GDP2017, .properties.GDP2018) DROP OTHERS;	
6. FILTER	<i>iq₃</i>
CASE WHERE KNOWN FUZZY SETS Decreasing_Population, Increasing_GDP CHECK FOR FUZZY SET Wanted USING Decreasing_Population AND Increasing_GDP ALPHA-CUT 0.8 ON Wanted DROP OTHERS;	
7. FILTER	
CASE WHERE WITH .properties.CountryCode, .properties.NUTSCode, .properties.NUTSLevel, .properties.NUTSName AND KNOWN FUZZY SETS Wanted GENERATE { .type : .type, .geometry : .geometry, .properties.CountryCode : .properties.CountryCode, .properties.NUTSCode : .properties.NUTSCode, .properties.NUTSLevel : .properties.NUTSLevel, .properties.NUTSName : .properties.NUTSName, .properties.rank : MEMBERSHIP_OF (Wanted) } DROP OTHERS;	

Specifically, the CASE WHERE condition, which is always true by construction, selects documents having the `.feature.item` field; the selected documents (i.e., all the documents in the collection, by construction) are restructured by the GENERATE action, in order to simplify them: all fields inside the `.feature.item` field are moved to the root level, so as the output documents have the fields named `type`, `property` and `geometry`.

The final DROP OTHERS clause is ineffective.

Apart from the name of the input collection (on line 1), the preamble is fixed and independent of internal queries within the *GeoSoft* query. We can say that it corresponds to the *get* function of Sect. 5.1, although to comply with the execution model of *J-CO-QL* it generates a collection of JSON documents.

Furthermore, notice that the `~fuzzysets` field (which is the counterpart of the `f.fuzzysets` map), has not been considered yet. In fact, it is automatically added and removed by the *J-CO-QL Engine*.

6.2 Core of the *J-CO-QL* Script

The *core* of the *J-CO-QL* script actually corresponds to the n internal queries $iq_1, \dots, iq_n$ of the *gsq GeoSoft* query. *J-CO-QL* instructions corresponding to each internal query are generated from the innermost iq_1 internal query to the outermost iq_n internal query. Listing 7 reports the core *J-CO-QL* script for the *GeoSoft* query presented in Listing 4.

- The FILTER instruction on line 4 corresponds to the innermost iq_1 internal query in Listing 4. For each document in the temporary collection (which cor-

responds to a feature in the input *GeoJSON* document), the instruction evaluates the membership degree to the *Decreasing_Population* fuzzy set. The soft condition expressed in the *WHERE* condition of q_1 is reported in the *USING* clause of the *FILTER* instruction. It exploits the fuzzy operator named *Has_Decreasing_Population* applied to the properties named, respectively, *.Pop2017* and *.Pop2018*. All property names are translated into the actual names of the corresponding fields, i.e., *.properties.Pop2017* and *.properties.Pop2018*, because feature properties are actually in the root-level *.properties* field.

The *CASE WHERE* condition selects documents in the input temporary collection having the fields named *.property.Pop2017* and *.property.Pop2018* (by means of the *WITH* predicate). The following *CHECK FOR FUZZY SET* clause *fuzzifies* the document, according to the *J-CO-QL* model presented in Sect. 3, generating the special *~fuzzysets* field that contains the field named *Decreasing_Population* whose value is the membership degree computed by the fuzzy operator *Has_Decreasing_Population*,

The final *DROP OTHERS* option is ineffective, since all documents in the input temporary collection are selected.

- The *FILTER* instruction on line 5 corresponds to the intermediate iq_2 internal query in Listing 4. Similarly to the *FILTER* instruction on line 4, it evaluates the membership degree of each document in the input temporary collection (the one generated by line 4) to the *Increasing_GDP* fuzzy set, by exploiting the *Has_Increasing_GDP* fuzzy operator. Notice again how the soft condition in the *WHERE* clause of iq_2 becomes the condition reported in the *USING* clause of the *FILTER* instruction.

We can observe that when a *GeoSoft* internal query keeps all properties (denoted as *SELECT **), one single *FILTER* instruction suffices in the *J-CO-QL* script.

- The outermost iq_3 internal query in Listing 4 projects features on a subset of properties. In this case, two *FILTER* instructions are necessary.

The first *FILTER* instruction is on line 6. Its goal is to evaluate the membership degree of each document in the input temporary collection (the one generated by line 5) to the *Wanted* fuzzy set. Again, the soft condition reported in the *WHERE* clause iq_3 is reported in the *USING* clause: notice that linguistic predicates are not translated, since they correspond to fuzzy-set names previously evaluated.

The *ALPHA-CUT* clauses in the outermost internal *GeoSoft* query has a direct counterpart in the *FILTER* instruction: only documents having a membership degree to the *Wanted* fuzzy set no less than 0.8 are selected.

The final *DROP OTHERS* option is ineffective, since all documents that are present in the input temporary collection are selected by the *CASE WHERE* clause.

- The *FILTER* instruction on line 7 is necessary to end processing iq_3 , because its *SELECT* clause projects features on a subset of properties and because the *ADD MEMBERSHIP OF* clause adds a new property to features.

Consequently, the *CASE WHERE* clause in the *FILTER* instruction selects those documents having the desired properties (translated into the actual structure of documents in the input temporary collection) and adds the *.rank* properties, by getting the membership degree to the *Wanted* fuzzy set through the *MEMBERSHIP_OF* function. In the *CASE WHERE* clause, notice the *KNOWN FUZZY SET* predicate:

it is a crisp predicate (in *J-CO-QL* the `WHERE` condition is a traditional Boolean condition, while the `USING` clause expresses soft conditions) that is true if the specified fuzzy sets have been already evaluated for the document (i.e., the fuzzy-set names appear as fields in the special `~fuzzysets` field).

The final `DROP OTHERS` option discards all documents (if any) that are not selected by the `CASE WHERE` clause.

Consequently, the patterns we applied to translate a *GeoSoft* internal query *iq* into a fragment of *J-CO-QL* script are the following ones:

- A `FILTER` instruction corresponds to the soft condition in the `WHERE` clause of *iq* and/or to the `ALPHA-CUT` clause.
The `CASE WHERE` clause selects documents having the properties that are referred to in *iq*;
the `CHECK FOR FUZZY SET` and `USING` clauses correspond to the `WHERE` clause in *iq*;
if present in *iq*, the `ALPHA-CUT` clause is present in the `FILTER` instruction.
- If the internal query *iq* projects features on a subset of properties and/or adds properties by means of the `ADD MEMBERSHIP OF` clause, a second `FILTER` instruction is present in the fragment of *J-CO-QL* script.
The `CASE WHERE` clause selects documents having properties specified in *iq*;
if the `ADD MEMBERSHIP OF` clause is in *iq*, the `CASE WHERE` clause is extended with the `KNOW FUZZY SETS` predicate;
the `GENERATE` action projects the `.properties` field of selected documents, with those listed in the `SELECT` clause in *iq* and the ones added by the `ADD MEMBERSHIP OF` clauses in *iq*.

6.3 Tail of the *J-CO-QL* Script

The *tail* of the *J-CO-QL* script implements the *set* function introduced to explain the semantic model of *GeoSoft* in Sect. 5.1. Its main goal is to aggregate documents so far selected (and currently in the temporary collection generated by the last instruction in the core of the script) into one single *GeoJSON* document. It also *de-fuzzifies* documents, sorts them if the `ORDER BY` clause is specified in the *GeoSoft* query and saves the output *GeoJSON* document into the database.

The tail of the *J-CO-QL* script is reported in Listing 7.

- The `FILTER` instruction on line 8 prepares documents to be grouped into the output *GeoJSON* document. By construction, the `CASE WHERE` condition is true for each document in the temporary collection, because all documents have the three specified fields. The `GENERATE` action inserts a fictitious key field, with a constant value into each document; this field will play the role of grouping key in the next `GROUP` instruction. The other fields are those required by the *GeoJSON* standard for features and are left untouched. The `DROPPING ALL FUZZY SETS` option *de-fuzzifies* the documents, i.e., removes the `~fuzzysets` field. The `DROP OTHERS` option is ineffective by construction. Remember that each single document in the output temporary collection is a feature to aggregate within the `.features` array field of the output *GeoJSON* document.

Listing 7. Translation of the *GeoSoft* query: tail.

```

8.  FILTER
    CASE WHERE WITH .type, .geometry, .properties
    GENERATE {
        .key      : 1,
        .type      : .type,
        .geometry  : .geometry,
        .properties : .properties
    }
    DROPPING ALL FUZZY SETS
    DROP OTHERS;

9.  GROUP
    PARTITION WITH .key
    BY .key INTO .features
    DROP GROUPING FIELDS
    ORDER BY .properties.rank DESC
    GENERATE {
        .type      : "FeatureCollection",
        .features  : .features
    }
    DROP OTHERS;

10. SAVE AS DetectedNUTS_Geojson@NUTS_Info;

```

- The GROUP instruction on line 9, rebuilds a unique *GeoJSON* document. First of all, it creates a partition of documents in the input temporary collection, in such a way that all documents are in one single partition; in fact, by means of the WITH predicate, all documents that hold a key field are selected. By means of the BY clause, documents in the partition are grouped based on the value of the key field: since we have only one constant value (generated by the FILTER instruction on line 8), all documents in the partition are grouped into one single document.

The INTO clause specifies the name of the new array field that has to contain grouped documents: in this case, the name is `.features` (to comply with the *GeoJSON* standard). The DROP GROUPING FIELDS option discards the key field. The ORDER BY clause sorts documents that have been grouped into the new `.features` array in descending order (DESC option) of values of the `.properties.rank` field, as specified in the ORDER BY clause in the *GeoSoft* query in Listing 4. The GENERATE action maintains the features field unchanged and adds a new type field with "FeatureCollection" as value, as required by the *GeoJSON* standard.

Notice that the new temporary collection now contains one lonely document that satisfies the *GeoJSON* standard.

- Finally, the SAVE AS instruction on line 10, saves the temporary collection holding the single *GeoJSON* document into the DetectedNUTS_Geojson collection, which is created into the NUTS_Info database as declared in the final SAVE AS clause of the *GeoSoft* query in Listing 4.

Figure 5b reports the output document obtained by the script, which derives from the input *GeoJSON* document reported in Fig. 5a that we built for the running example.

As a result, we can state that the *GeoSoft* query can be actually automatically translated into a *J-CO-QL* script. This also proves that the *J-CO* Framework can be actually exploited as execution engine of domain-specific query languages designed to work on specific *JSON* formats, such as *GeoJSON*.

7 Conclusions

In this paper, we introduced *GeoSoft*, a domain-specific language for querying features described in a *GeoJSON* document. The paper introduced the language and the *J-CO* Framework, which we used as the execution environment for *GeoSoft* queries. The *J-CO* Framework is under development at University of Bergamo to be an advanced tool to integrate, manipulate and query possibly-heterogeneous collections of *JSON* documents, by means of its query language named *J-CO-QL*. In this paper, we demonstrate the feasibility of the *GeoSoft* language, by showing how it is possible to automatically translate *GeoSoft* queries into *J-CO-QL* scripts; indeed, scripts reported in Listings 5, 6 and 7 were generated by the prototype *GeoSoft* interpreter we recently built.

As a future work, we will continue developing the *GeoSoft* language, in order to provide users with sophisticated functionalities able to deal with geometries and to integrate features coming from several *GeoJSON* documents.

Acknowledgment. We warmly thank Luca Assolari, student of the Master Degree in Computer Science at University of Bergamo (Italy), who implemented the prototype *GeoSoft* interpreter.

```

geoSoft      : ( selectClause ( orderBy )? saveAs ';' ) * ;
selectClause : 'SELECT' properties
              from
              ( where )?
              ( alphaCut ) *
              ( addMembership ) * ;

properties   : '*' | property ( ',' property ) * ;
property     : DOT_ID ( 'AS' DOT_ID )? | expression 'AS' DOT_ID ;
from         : 'FROM' source ( ',' source ) * ;
source       : ID ( '@' ID )? | '(' select ')' ;
where        : 'WHERE' condition ( 'FOR' 'FUZZY' 'SET' ID )? ;
alphaCut     : 'ALPHA-CUT' ID ':' number ;
addMembership : 'ADD' 'MEMBERSHIP' 'OF' ID 'AS' DOT_ID ;
orderBy      : 'ORDER' 'BY' DOT_ID ( 'ASC' | 'DESC' )?
              ( ',' DOT_ID ( 'ASC' | 'DESC' )? ) * ;
saveAs       : 'SAVE' 'AS' ID ( '@' ID )? ;

```

Fig. 7. *GeoSoft* grammar.

A Syntax of *GeoSoft*

In Fig. 7, we present the syntax (grammar) of the *GeoSoft* language introduced in Sect. 5. The grammar is formulated in EBNF¹⁶ notation according to the convention applied by ANTLR (*ANother Tool for Language Recognition*)¹⁷. ANTLR is a widely-known parser generator that denotes non-terminal elements (rules) in lower case, while

¹⁶ https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form, accessed on 30/10/2021.

¹⁷ <https://www.antlr.org/>, accessed on 30/10/2021.

terminal elements (tokens) are in upper case or directly declared between quotes. The *geoSoft* rule is the starting rule of the grammar.

For the sake of simplicity, we do not include the definitions of the `condition`, `expression` and `number` rules, but their meaning is denoted by their names. The `ID` and `DOT_ID` tokens denote the classic `identifiers`, respectively, not having (`ID`) or having (`DOT_ID`) a dot character as starting character.

References

1. Aufaure, M.A., Trépied, C.: A survey of query languages for geographic information systems, p. 3 (1996)
2. Bieber, M., Kacmar, C.: Designing hypertext support for computational applications. *Commun. ACM* **38**(8), 99–107 (1995)
3. Bordogna, G., Capelli, S., Ciriello, D.E., Psaila, G.: A cross-analysis framework for multi-source volunteered, crowdsourced, and authoritative geographic information: the case study of volunteered personal traces analysis against transport network data. *Geo-Spat. Inf. Sci.* **21**(3), 257–271 (2018)
4. Bordogna, G., Capelli, S., Psaila, G.: A big geo data query framework to correlate open data with social network geotagged posts. In: Bregt, A., Sarjakoski, T., van Lammeren, R., Rip, F. (eds.) *GIScience 2017. LNCS*, pp. 185–203. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-56759-4_11
5. Bordogna, G., Psaila, G.: Soft aggregation in flexible databases querying based on the vector p-norm. *Int. J. Uncertain. Fuzziness Knowl.-Based Syst.* **17**(supp01), 25–40 (2009)
6. Bordogna, G., Psaila, G.: Customizable flexible querying in classical relational databases. In: *Handbook of Research on Fuzzy Information Processing in Databases*, pp. 191–217. IGI Global (2008)
7. Bosc, P., Pivert, O.: SQLf: a relational database language for fuzzy querying. *IEEE Trans. Fuzzy Syst.* **3**(1), 1–17 (1995)
8. Garcia Bringas, P., Pastor, I., Psaila, G.: Can blockchain technology provide information systems with trusted database? The case of hyperledger fabric. In: Cuzzocrea, A., Greco, S., Larsen, H.L., Saccà, D., Andreassen, T., Christiansen, H. (eds.) *FQAS 2019. LNCS (LNAI)*, vol. 11529, pp. 265–277. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-27629-4_25
9. Butler, H., et al.: The GeoJSON format. Internet Engineering Task Force (IETF) (2016)
10. Cai, G.: GeoVSM: an integrated retrieval model for geographic information. In: Egenhofer, M.J., Mark, D.M. (eds.) *GIScience 2002. LNCS*, vol. 2478, pp. 65–79. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45799-2_5
11. Cattell, R.: Scalable SQL and NoSQL data stores. *ACM SIGMOD Rec.* **39**(4), 12–27 (2011)
12. Costagliola, G., Tortora, G., Tucci, M., Busillo, M.: GISQL—a query language interpreter for geographical information systems. In: Spaccapietra, S., Jain, R. (eds.) *Visual Database Systems 3. ITIFIP*, pp. 275–286. Springer, Boston, MA (1995). https://doi.org/10.1007/978-0-387-34905-3_17
13. Egenhofer, M.J.: Spatial SQL: a query and presentation language. *IEEE Trans. Knowl. Data Eng.* **6**(1), 86–95 (1994)
14. Florescu, D., Fourny, G.: JSONiq: the history of a query language. *IEEE Internet Comput.* **17**(5), 86–90 (2013)
15. Formica, A., Mazzei, M., Pourabbas, E., Rafanelli, M.: Querying distributed GIS with GeoPQLJ based on GeoJSON, pp. 175–182 (2019)

16. Fosci, P., Marrara, S., Psaila, G.: Soft querying GeoJSON documents within the J-CO framework. In: 16th International Conference on Web Information Systems and Technologies (WEBIST 2020), pp. 253–265. SCITEPRESS-Science and Technology Publications, Lda. (2020)
17. Fosci, P., Psaila, G.: *J-CO*, a framework for fuzzy querying collections of *JSON* documents (demo). In: Andreassen, T., De Tré, G., Kacprzyk, J., Legind Larsen, H., Bordogna, G., Zadrozny, S. (eds.) FQAS 2021. LNCS (LNAI), vol. 12871, pp. 142–153. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86967-0_11
18. Fosci, P., Psaila, G.: Powering soft querying in J-CO-QL with JavaScript functions. In: Sanjurjo González, H., Pastor López, I., García Bringas, P., Quintián, H., Corchado, E. (eds.) SOCO 2021. AISC, vol. 1401, pp. 207–221. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-87869-6_20
19. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of JSON data sets through fuzzy sets: a case study. *Information* **12**(7), 258 (2021)
20. Galindo, J., Urrutia, A., Piattini, M.: *Fuzzy Databases: Modeling, Design, and Implementation*. IGI Global (2006)
21. Anderson, J.C., Lehnardt, J., Slater, N.: *CouchDB: The Definitive Guide*. O'Reilly Media, Inc. (2010)
22. Jacobs, B.E., Walczak, C.A.: A generalized query-by-example data manipulation language based on database logic. *IEEE Trans. Softw. Eng.* **1**, 40–57 (1983)
23. Kim, H.J., Korth, H.F., Silberschatz, A.: Picasso: a graphical query language. *Softw. Pract. Exp.* **18**(3), 169–203 (1988)
24. Mayer, B.: Beyond icons: towards new metaphors for visual query languages for spatial information systems. In: Cooper, R. (ed.) IDS 1992. Workshops in Computing, pp. 113–135. Springer, Glasgow (1992). https://doi.org/10.1007/978-1-4471-3423-7_8
25. Nayak, A., Poriya, A., Poojary, D.: Type of NoSQL databases and its comparison with relational databases. *Int. J. Appl. Inf. Syst.* **5**(4), 16–19 (2013)
26. Ong, K.W., Papakonstantinou, Y., Vernoux, R.: The SQL++ unifying semi-structured query language, and an expressiveness benchmark of SQL-on-Hadoop, NoSQL and NewSQL databases. CoRR, abs/1405.3631 (2014)
27. Psaila, G., Fosci, P.: Toward an analyst-oriented polystore framework for processing json geo-data. In: International Conferences on Applied Computing 2018, Budapest, Hungary, 21–23 October 2018, pp. 213–222. IADIS (2018)
28. Psaila, G., Fosci, P.: J-CO: a platform-independent framework for managing geo-referenced JSON data sets. *Electronics* **10**(5), 621 (2021)
29. Psaila, G., Marrara, S.: A first step towards a fuzzy framework for analyzing collections of JSON documents. In: IADIS AC 2019, pp. 19–28 (2019)
30. Staes, F., Tarantino, L., Tiems, A.: A graphical query language for object oriented databases. In: Proceedings 1991 IEEE Workshop on Visual Languages, pp. 205–210 (1991)
31. Zadeh, L.A.: The concept of a linguistic variable and its application to approximate reasoning—I. *Inf. Sci.* **8**(3), 199–249 (1975)